

1 Viscous Burgers' Equation

This project implements and verifies a space-time finite element method (FEM) for solving the one-dimensional viscous Burgers' equation. We use bilinear rectangular elements over a structured spacetime grid, impose Dirichlet boundary and initial conditions, and compare the numerical solution to the known analytical one. We demonstrate second-order convergence in the L^2 norm and confirm solver accuracy and stability using PETSc. We found this specific example on numerical-mooc, which is a Github repository with self-guided tutorials for solving PDEs with numerical methods.

1.1 Mathematical Model

We consider the viscous ($\nu > 0$) nonlinear Burgers' equation in one-dimension:

$$u_t + uu_x = \nu u_{xx} \quad x \in (0, 2\pi) \quad (1)$$

The initial conditions,

$$u = -\frac{2\nu}{\psi} \psi_x + 4$$
$$\psi(t=0) = \psi_0 = \exp\left(\frac{-x^2}{4\nu}\right) + \exp\left(\frac{-(x-2\pi)^2}{4\nu}\right),$$

with an analytical solution,

$$u = -\frac{2\nu}{\psi} \psi_x + 4 \quad (2)$$
$$\psi = \exp\left(\frac{-(x-4t)^2}{4\nu(t+1)}\right) + \exp\left(\frac{-(x-4t-2\pi)^2}{4\nu(t+1)}\right).$$

Periodic spatial boundary conditions are employed such that $u(0) = u(2\pi)$

1.2 Space-Time Finite Element Discretization

1.2.1 Variational Form

We multiply the PDE by a test function ϕ and integrate over the spacetime domain to obtain the weak form equation:

$$\int \int (u_t \phi + uu_x \phi - \nu u_{xx} \phi) \, dx \, dt = 0$$
$$\int \int (u_t \phi + \nu u_x \phi_x + uu_x \phi) \, dx \, dt = 0 \quad (3)$$

You may notice that in going from the strong form, Eq. 1, to the weak form, Eq. 3, we integrated by parts spatially. This moved a derivative over to the test function, and since the spatial BC are periodic, we were able to drop the boundary term from the weak form.

We approximate u using basis functions over rectangular elements. With $\xi = x/hx$ and $\tau = t/ht$, we define the rectangular basis functions:

$$\phi_1(\xi, \tau) = (1 - \xi)(1 - \tau),$$

$$\phi_2(\xi, \tau) = \xi(1 - \tau),$$

$$\phi_3(\xi, \tau) = \xi\tau,$$

$$\phi_4(\xi, \tau) = (1 - \xi)\tau.$$

We assume that u can be approximated over each element in the domain by a linear combination of the basis functions. This assumption is reasonable given that the size of the rectangular elements is sufficiently small.

$$u_{(e)} \approx \sum_{i=1}^4 a_i \phi_i$$

1.2.2 Element Stiffness Matrix

The local element matrix (for the linear terms) is computed symbolically and hardcoded using expressions dependent on grid spacing h :

$$A_{ij}^{(e)} = \int_0^{h_t} \int_0^{h_x} (\partial_t(\phi_i)\phi_j + \nu\partial_x(\phi_i)\partial_x(\phi_j)) dx dt + \int_0^{h_t} \int_0^{h_x} N_{ij}^{(e)}(u^*) dx dt$$

For the nonlinear terms, $N_{ij}^{(e)}$, we use Gaussian quadrature to update the integral as necessary using the current u^* iterate.

The result is a 4×4 matrix with entries involving h and constants. This matrix is reused for all elements due to uniformity. The symbolic calculation of the element stiffness matrices for the linear terms utilizes Python's SymPy.

1.3 Global Assembly and Solution

We loop over all elements, assemble the global stiffness matrix and residual vector, and solve using PETSc's SNES nonlinear solver. For initial testing purposes, a direct LU solver is used via command-line options:

```

###----- LU command line options ----- #
srun --mpi=pmix_v3 -n ${NTASKS} \
  ./Burgers_1p1_PC \
  -da_processors_x 1 \
  -dm_view \
  -snes_type newtonls \
  -snes_linesearch_type bt \
  -snes_monitor \
  -snes_rtol 1e-10 \
  -snes_atol 1e-12 \
  -ksp_type preonly \
  -pc_type lu \
  -pc_factor_mat_solver_type mumps \
  -use_shell_pc 0 \
  -slab_pc_nonlinear 0 \
  -slab_pc_ras 0 \
  -ksp_monitor_true_residual -ksp_converged_reason \
  -IC 1 \
  -t0 0.0 -tF 0.563 \
  -nx 400 -nt 400 \
  -viscosity 0.07 \
  -overlap 0 \
  -interface_width 0 \
  -memory_view \
  -log_view :Burgers_testing/PC_1p1_400x400_LU_10ranks_log-view.txt

```

1.4 Boundary and Initial Condition Enforcement

We employ spatial periodic boundary conditions. These are mainly handled internally by PETSc's DMDA framework. Our initial conditions are implemented by enforcing Dirichlet boundary conditions on the left temporal boundary. We do this by

- Zeroing the corresponding rows of the Jacobian.

- Inserting a 1 on the diagonal.
- Setting the residual to its current entry minus the initial condition.

1.5 Error Computation and Convergence Rate

The discrete L^2 error norm is computed as:

$$E = \|u_h - u\|_{L^2} = \left(\sum_{i,j} (u_h(x_i, t_j) - u_{\text{exact}}(x_i, t_j))^2 h_x h_t \right)^{1/2},$$

where h_x and h_t are the spatially and temporal mesh size (distance between space and time nodes). We utilize a structured mesh, so these are constant throughout the mesh. u_{exact} is our analytical solution, u_h is our numerical solution.

Given two successively refined grids (h_i, E_i) and (h_{i+1}, E_{i+1}) , we find the order of convergence,

$$p_i = \frac{\log(E_i/E_{i+1})}{\log(h_i/h_{i+1})}.$$

With finer refinements, we expect the order of convergence to approach 2nd order. This is because of how we chose to sample 4 nodes per element through our 4 basis functions. To increase the order of convergence of the scheme, we would need to sample more nodes on each element.

1.6 Results (Direct LU Solve)

1.6.1 Results with $\nu = 0.07$

We ran the simulation for increasing values of $nx = nt$, and measured the L^2 error norm. By doubling the number of nodes in space and time, we halve the distance between space and time nodes such that $h_i/h_{i+1} = 2$ for each i for h_x and h_t .

$nx = nt$	L^2 Error Norm	Rate
100	4.564×10^{-2}	-
200	9.317×10^{-3}	2.29
400	2.284×10^{-3}	2.03
800	5.697×10^{-4}	2.00

Table 1: All runs were done with $\nu = 0.07$ on a physical domain $\Omega \times T = [0, 2\pi] \times [0, 0.563]$, matching the tests done at numerical-mooc.

The convergence rate seems to be approaching 2nd order. The images also indicate that our numerical solution is a good approximation to the analytical one, Figs. 1 and 2.

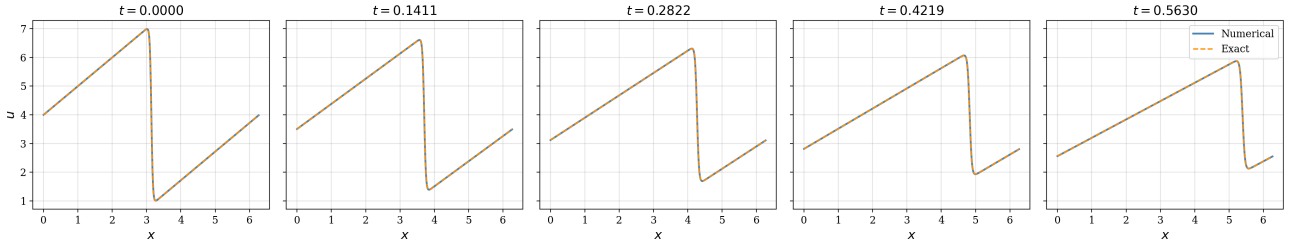


Figure 1: Visual comparison between numerical and analytical solutions at some varying timeslices of the viscous Burgers' equation for $nx = nt = 400$.

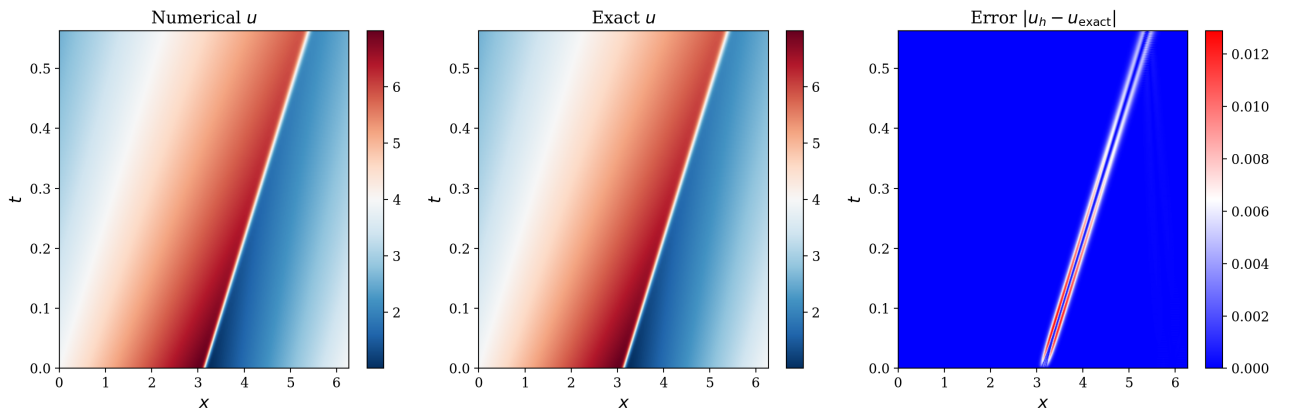


Figure 2: Comparison of the viscous Burgers' equation numerical and analytical solution over the whole spacetime domain. The rightmost plot shows us that our solution is least accurate near the 'shock,' which is to be expected.

1.6.2 Results with $\nu = 0.01$

We also decrease ν from 0.07 to $\nu = 0.01$ and perform a convergence study. We see that with ν closer to 0, the sharp change in the solution u is much harder to resolve, as expected. Table 6 and figures 4 and 3 show these results.

$nx = nt$	L^2 Error Norm	Rate
100	6.80×10^{-1}	-
200	2.596×10^{-1}	1.39
400	7.152×10^{-2}	1.86
800	1.324×10^{-2}	2.43
1600	2.89×10^{-3}	2.20

Table 2: All runs were done with $\nu = 0.01$ on a physical domain $\Omega \times T = [0, 2\pi] \times [0, 0.563]$.

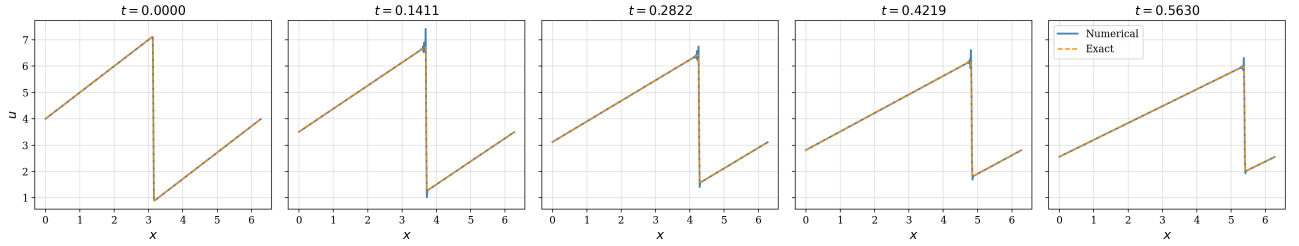


Figure 3: Visual comparison between numerical and analytical solutions at some varying timeslices of the viscous Burgers' equation with $\nu = 0.01$ for $nx = nt = 400$.

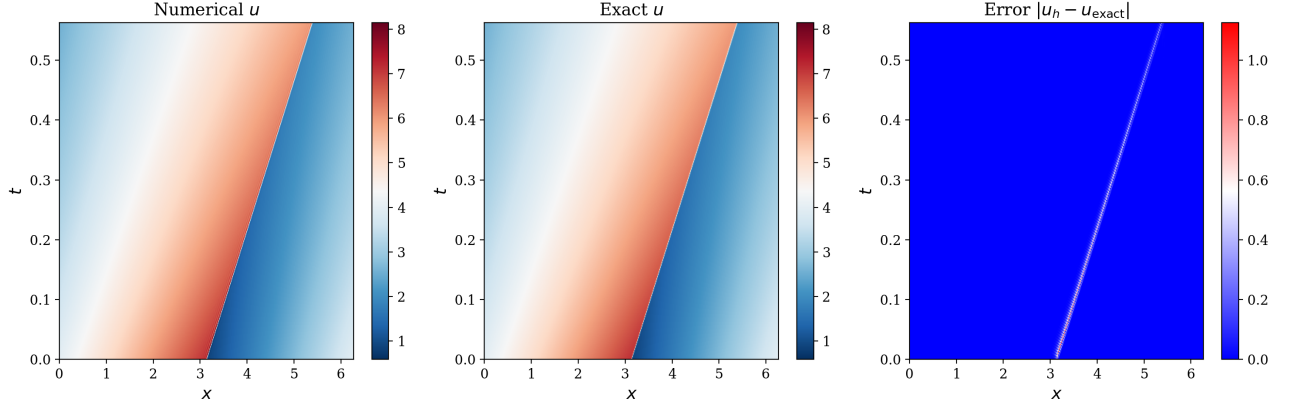


Figure 4: Comparison of the viscous Burgers' equation numerical and analytical solution over the whole spacetime domain. The rightmost plot shows us that our solution is least accurate near the 'shock,' which is to be expected.

1.7 Conclusion

We implemented a spacetime finite element method for the 1D viscous Burgers' equation using PETSc and verified it against an analytical solution. The method demonstrates second-order convergence in the L^2 norm. We note that the near discontinuity becomes more difficult to resolve as ν approaches 0.0, which is expected. Future work will investigate adaptive mesh refinement in space along the pseudo-shock region.

This work was done with a direct LU solver and no preconditioner applied. The next chapter will rerun these convergence tests using our time-decomposition additive Schwarz preconditioning scheme. We will investigate how our PC holds up as $\nu \rightarrow 0$. Then, we will look at the inviscid Burgers' equation ($\nu = 0$) to investigate the behavior of our spacetime formulation and PC scheme with a true shock.

2 RAS Preconditioning Results

2.1 Results with $\nu = 0.07$

We find that RAS does properly precondition the system. Table 3 shows a convergence study indicating 2nd order convergence.

$nx = nt$	L^2 Error Norm	Rate
100	4.564×10^{-2}	-
200	9.317×10^{-3}	2.29
400	2.284×10^{-3}	2.03
800	5.697×10^{-4}	2.00

Table 3: All runs were done with $\nu = 0.07$ on a physical domain $\Omega \times T = [0, 2\pi] \times [0, 0.563]$.

We also find that including nonlinear updates in the PC, rather than just using a frozen-in preconditioner which only accounts for the linear terms, allows for much better results, Table 4. Both methods (frozen PC vs nonlinear updated PC) produced the same L2 error norm values. We see that updating the PC with the nonlinear contributions at each new iterate drastically decreases the required number of GMRES iterations. This is because the linear frozen PC is not a good enough approximation to the true Jacobian.

We note that the nonlinear updates were not needed for good results in the coupled nKG problem. This seems to indicate that we will need to include the nonlinear updates for shock problems. In this document, unless otherwise stated, we use RAS with nonlinear updates. This means that after each SNES iteration, the current solution iterate u_k is used to update the nonlinear terms in the stiffness matrix produced for each slab.

Table 5 compares RAS to LU when $\nu = 0.07$. We see that RAS requires similar memory and less time than LU. However, if we increase the problem size too much, LU becomes faster than RAS. We show that lagging the nonlinear updates in the PC helps this issue. We tested lags of 3-6. This means that we only update the nonlinear terms in the PC every 3-6 SNES iterations as opposed to doing it every single iteration. We find that using a lag of 5 results in the most desirable behavior. Note that lagging does increase the memory consumption because now convergence requires more GMRES iterations since the PC is not as good of an approximation at every Newton step, but it is much faster than rebuilding and solving the stiffness matrix every single Newton step. Unless otherwise stated, assume that no lagging was used.

Table 4: Comparison of frozen-in and nonlinearly updated RAS PC.

Method	$nx = nt$	Memory (GB)	Time (s)	GMRES Iters
Nonlinear	100	0.781	0.384	195
Frozen		0.790	1.963	4314
Nonlinear	200	0.854	2.596	213
Frozen		0.892	13.96	5041
Nonlinear	400	1.08	19.41	197
Frozen		1.31	73.39	5237
Nonlinear	800	2.21	169.8	166
Frozen		3.02	273.4	5007

Table 5: Comparison of LU and RAS. 4 processors were used

Method	$nx = nt$	Memory (GB)	Time (s)	GMRES Iters
LU	100	0.791	0.858	-
RAS		0.781	0.384	195
LU	200	0.884	7.89	-
RAS		0.854	2.596	213
LU	400	1.15	68.97	-
RAS		1.08	19.41	197
LU	800	2.25	527.1	-
RAS		2.21	169.8	166
LU	1600	6.95	1128.0	-
RAS		6.68	2089.0 no lag	165
LU	1600	6.95	1128.0	-
RAS		9.15	474.7 5 lag	610

2.2 Results with $\nu = 0.01$

We test robustness of our PC as we decrease $\nu = 0.07 \rightarrow \nu = 0.01$. Again, we see that our PC properly preconditions the system. This is promising as we move into the inviscid Burgers' equation, where we investigate our spacetime FEM discretization and RAS PC on a problem with a true shock.

$nx = nt$	L^2 Error Norm	Rate
100	6.80×10^{-1}	-
200	2.596×10^{-1}	1.39
400	7.152×10^{-2}	1.86
800	1.324×10^{-2}	2.43
1600	2.891×10^{-3}	2.20

Table 6: All runs were done with $\nu = 0.01$ on a physical domain $\Omega \times T = [0, 2\pi] \times [0, 0.563]$ and 1 lag.